

Predictions to the Water Level of Changjiang Waterway Employing Deep Learning Algorithm LSTM

Li Youwei

Changjiang Waterway Planning, Design and Research Institute, Wuhan, China

Jiang Zhaobing

School of Civil Engineering, Sanjiang University, Nanjing, China

Tan Kun

Changjiang Waterway Planning, Design and Research Institute, Wuhan, China

ABSTRACT

As an essential part of a coast, waterways undergo dynamic transformations, qualitatively and quantitatively. The most dynamic transformation among all is the water-level fluctuations (WLFs), affecting major coastal traffic regulations. Until today, worldwide WLFs focus on large lakes for their significance in globalization. However, forecasting WLFs of artificial/non-artificial waterways have tremendous economic benefits for locals. Forecasting WLFs can be seen as a time series problem. Artificial neural networks (ANNs) have shown promises of time series forecast and are well-documented. This study used the Changjiang waterway's water level data to make time series forecasts using deep learning algorithm Long Short-Term Memory (LSTM). An essential task that artificial neural networks can perform is to discover complex temporal dependence and that makes neural network an applicable approach to time series forecast. Naive model was constructed as a baseline model to measure the improvement and the validity of multilayer perceptron model. Univariate and multivariate adaptive models were then constructed by manipulations of the data. Both short-term and long-term predictions showed promises of multilayer perceptron model in time series forecasting and their performances were measured by root mean square error (RMSE) and time complexities.

KEY WORDS: Waterway water-level fluctuations; deep learning; LSTM; Time Series.

INTRODUCTION

As an essential part of a coast, waterway undergoes dynamic transformations, qualitatively and quantitatively. The most dynamic transformation among all is the water-level fluctuations (WLFs), affecting major waterway regulations (Jonkeren et al. 2011). Until today, worldwide WLFs focus on large lakes for their significance in globalization, for instance, Lake Van (Altunkaynak et al. 2003). Forecasting WLFs of artificial/non-artificial local waterways may have tremendous economic benefits for local governments. Forecasting WLFs measured daily can be seen as a time series problem. Artificial neural networks (ANNs) have shown promises of time series forecast in surface water-level forecasting (Piasecki et al. 2017) and are well-documented.

This will be the first time investigating in the problem of waterways' WLFs using ANNs, or more specifically, the development of LSTM models in Changjiang waterway.

Due to all the merits of neural networks, in this study, we will be investigating the problem of navigable rivers WLF using a basic, fully-connected, and self-adaptive, neural network framework, or more specifically, the development of adaptive LSTM in Changjiang navigable river. The dataset obtained from Changjiang Waterway Planning, Design and Research Institute consists of the water level measurements from a total of ten cities in China from 2010 to 2019. Being adaptive means the NN model will be rebuilt by quickly adjusting its weights from the feedback of the new data, generating adapted forecasts, resulting in more accuracy es experimented in this study. This adaptive nature of the model is indispensable and highly valuable in real life as its ability to deal with the unpredictability. For instance, if there are some factors heavily influencing today's water-level, an adaptive model can quickly gain information and adapt to this change, reconstructing the model for future accurate predictions.

METHODS

Problem Definition

In this study, we are interested in discovering the long-term trend and accurate short-term forecasts for water-level fluctuations in Changjiang waterway, shown as Fig. 1. A reasonable branching of the problem can be: 1. Univariate multi-step forecast for water level of Changjiang waterway; 2. Multivariate multi-step forecast for water level of Changjiang waterway. A typical dataset is shown at the first row in Fig.1, i.e., the Nanjing waterway station. A general workflow, shown as Fig. 2, of our study can be structured base on this division, ranging from problem definition to final deliverable model. The former model investigates Changjiang's own temporal dependencies, whereas the latter model investigates the interactions between waterways as other waterways, shown as Fig. 3, are upstream or downstream of the Changjiang waterway. From the heatmap constructed (Fig. 4), we can see that Changsha's water-level is less correlated with other waterways which make sense from a geological aspect and on the other hand, other cities have shown close correlation to the city of interest.

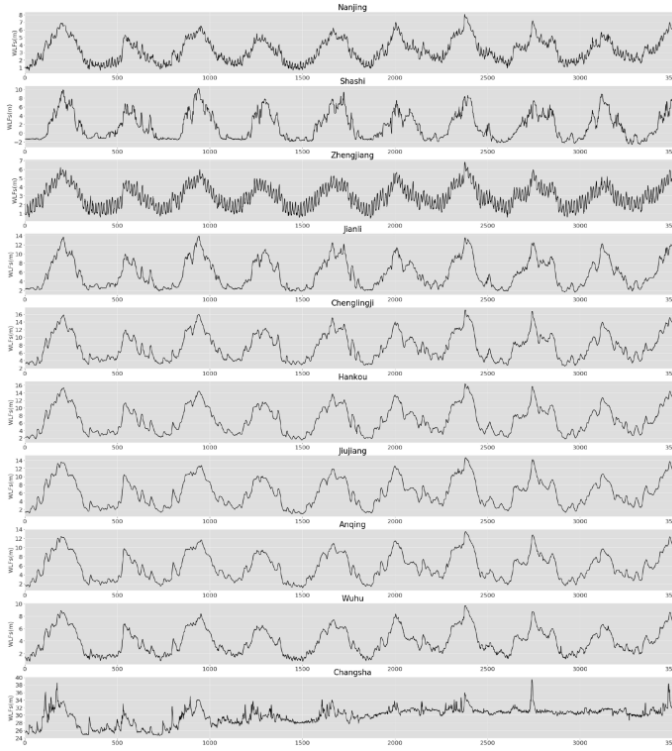


Fig. 1 Changjiang Waterway water-level measurements from 2010/01/01 to 2019/09/15

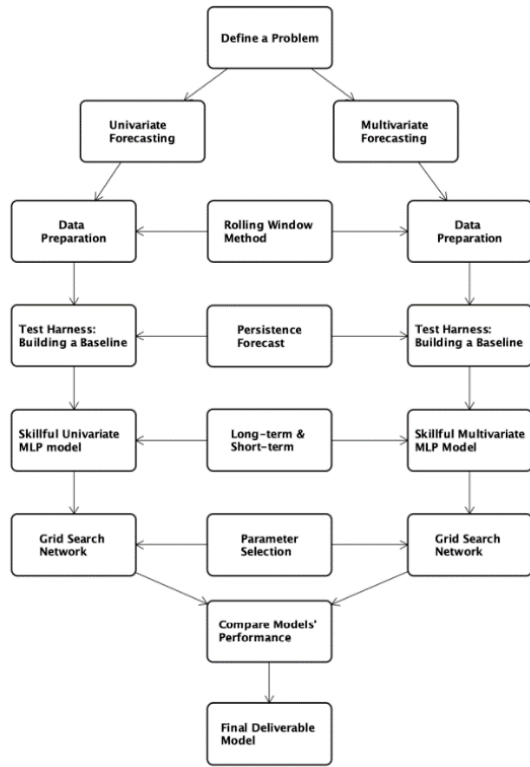


Fig. 2 A general workflow of Changjiang's waterway time series forecast

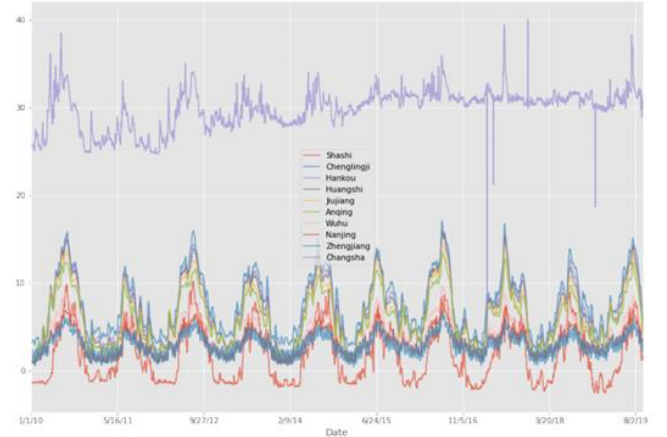


Fig. 3 All waterways' measurements from 2010/01/01 to 2019/09/15.

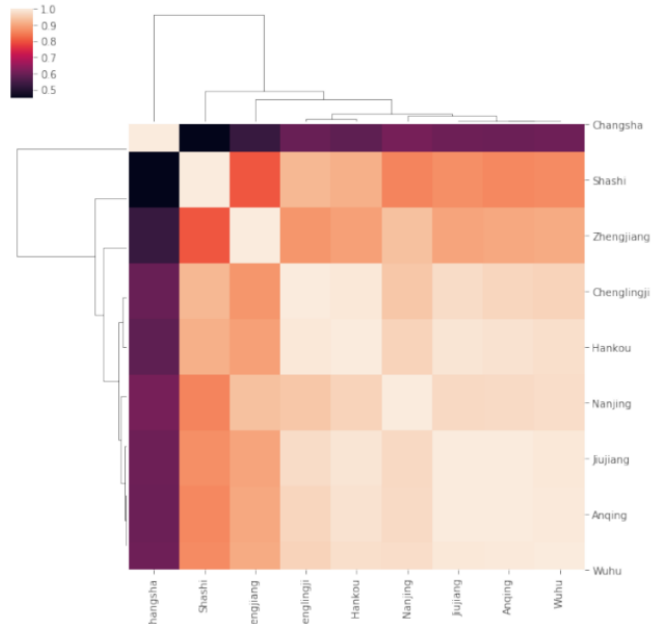


Fig. 4 All waterways' heatmap

LSTM: Long Short-Term Memory

LSTM, was colloquially referred to as the 'vanilla' neural network (Hastie et al. 2009), is a branch of feed-forward networks (Svozil et al. 1997) and it's one of the first developed ANNs. LSTM usually contains multiple layers. It mainly consists of three layers: input layer, hidden layer and output layer (Rosenblatt 1961). The number of neurons (Fig. 5) in each layer and the number of layers are often hyperparameters and can be tuned later on to improve performance. In this study, in the output layer, the number of neurons in an output layer is equal to the number of dependent variables after the window method. Each neuron contains bias(b), input(x) and weight(w) as three main components and it also has a non-linear activation function that produces a cumulative output of the preceding neurons. Each output of a single neuron can be written as:

$$y_i = \sigma \left(\sum_i w_i x_i + b_i \right) \quad (1)$$

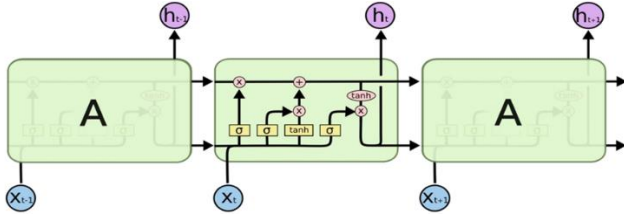


Fig. 5 neuron model which may be located in the hidden and output layers

Whereas σ is an activation function. Common choices of activation function in modern deep learning consists of:

$$\begin{aligned} \text{sigmoid: } \sigma(z) &= \frac{1}{1 + e^{-z}} \\ \text{hyperbolic tangent: } \sigma(z) &= \frac{e^z - e^{-z}}{e^z + e^{-z}} \\ \text{ReLU: } \sigma(z) &= \max(0, z) \\ \text{Leaky-Relu: } \sigma(z) &= \begin{cases} x & \text{if } x > 0 \\ 0.01x & \text{otherwise} \end{cases} \end{aligned} \quad (2)$$

In this study, ReLu (rectified linear unit) was tested and selected over sigmoid and hyperbolic tangent functions as its ability to deal with the Vanishing Gradient Problem making it one of the most popular activation functions (Ramachandran et al. 2017). LSTM is a gradient-descent based learning method; common selection such as hyperbolic tangent functions have gradients between $(1, -1)$. As gradient-descent goes on, the slope and gradients of function decreases exponentially, resulting in slow learning and it is computationally costly (Goodfellow et al. 2016).

LSTM utilizes supervised learning technique called back propagation (Hecht-Nielsen 1992) for training (Fig. 6). Back propagation allows the information the cost function to flow backward to the network in order to computer gradients which is a fairly simple and inexpensive procedure (Goodfellow et al. 2016).

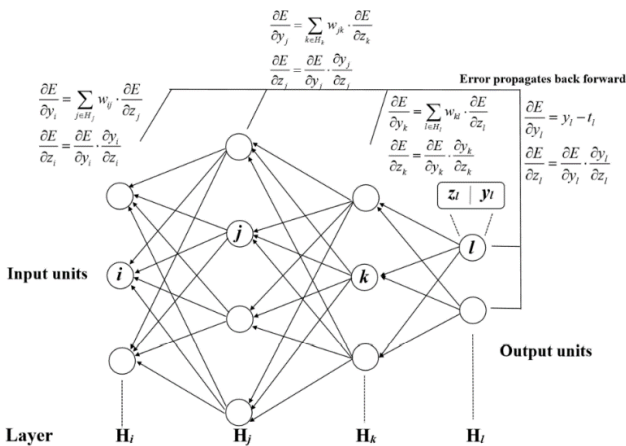


Fig. 6 a simple back propagation for gradient descent

Its learning occurs between perceptions by updating connecting weights W . The error of each output node can be expressed as,

$$e_i = (\hat{y}_i - y_i) \quad (3)$$

and root mean squared error (RMSE) will be used as the metrics to measure the overall performance of the model

$$e = \sqrt{\sum_{i=1}^n (\hat{y}_i - y_i)^2 / n} \quad (4)$$

To minimize e , gradient descent is applied to update the weights(w):

$$w_{ij}(n+1) = w_{ij}(n) - \eta \frac{\partial e(n)}{\partial V_j(n)} * y_i(n) \quad (5)$$

where y_i is the output of the preceding neuron, v_j is the local field and η is the learning rate which 96 is to ensure the quick converge to a good model(Haykin 1994).

LSTM has advantages and disadvantages in different settings. An issue that LSTM encounters is as the number of input features increases, the algorithm will become inefficient due to its fully connected nature (Driss et al. 2017). CNN was then developed because of the issue using shared weights between neurons. In Changjiang's waterway data, because the input size is fairly small, it's reasonable to construct a LSTM model because the small number of total parameters. The initial dataset was cleaned by imputing miss-labeled points and divided into training (3215 days) and testing (330 days) sets by its time steps, i.e. using the oldest 3215 days or less to train the model, and the latest 330 days or less to test the model. Next, the data need to be prepared for the model constructions.

Data Preparation: Window method

The use of previous time steps to predict the next time steps is called the rolling window method (Li et al. 2014), in short, the window method or lag method. It's a common method in statistics and time series analysis. The number of previous time steps is called window width or lag size (Bontempi et al. 2012). Window method is the foundation of turning time series problem into supervised learning problem. We will apply the window method to re-frame our data for univariate and multivariate LSTM models.

Suppose we have a week of data and each day (observation) has one corresponding value and we want to construct a model that has the ability to predict the next two days, i.e., day8 and day9. To do that, reframing the data using previous three days, i.e. window size of 3, to predict the next two days, meaning using the values from day 1, 2, 3 as X s and values from day 4, 5 as corresponding Y s, using the values from day 2, 3, 4 as X s and values from day 5, 6 as corresponding X s and etc. (As demonstrated in Fig. 7) The resulting matrix will have one row that contains only one Y , as our data is relatively abundant, we can delete this row as it will not impair our models' performance. For multivariate data, the window method can be analogously applied, treat each value S_i as a set of values (Tsay 2013).

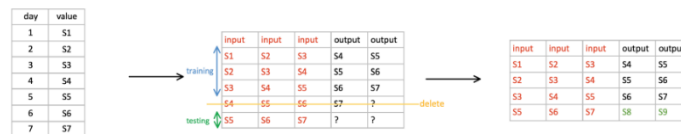


Fig. 7 window method used on a mock univariate data

Applying this technique on the real dataset with shape [3545, 10], i.e., water-level measurements of 3545 days in ten cities in china is straightforward. For brevity, we will illustrate this process for a seven-day short-time forecast, i.e., $N_{output} = 7$, (Fig. 8). First of all, split the data into training set 129 and testing set by time steps, resulting in two data-frames with shape [3213, 10] and [329, 10] (the 130 values 3213 and 329 are chosen because they are divisible by 7). Then restructure the both training 131 set and testing set into windows of 7, resulting in two 3-D data-frames with shape [459, 7, 10] and [47, 7, 10]. Additionally, use Window Method to transform the restructured 3-D training set into a data-frame for supervised learning. The window size, i.e. the number of previous time steps to predict the next time steps, is a parameter that can be tuned later on; for now, we will call it N_{output} . If only using Changjiang's water-level data, i.e. univariate forecasting, the independent variables, X s, will be shaped as $[\psi(N_{input}, N_{output}), N_{input}, 1]$ where ψ is a value varies with N_{input} and N_{output} (7 in this case); whereas the dependent variables, Y s will be shaped as $[\psi(N_{input}, N_{output}), N_{output} = 7, 1]$. $\psi(N_{input}, N_{output})$ can be easily derived as,

$$\psi(N_{input}, N_{output}) = 3214 - N_{input} - N_{output} \quad (6)$$

where $3214 = \text{length of the training set} (=3213) + 1$. For instance, if $N_{input} = 7$, $\psi(N_{input}, N_{output}) = 3214 - 7 - 7 = 3200$ from the window method. On the other hand, if using all 10 cities to 142 predict Changjiang's water-level, i.e. multivariate forecasting, the independent variables, X s, will be shaped as $[\psi(N_{input}, N_{output}), N_{input}, 10]$; whereas the dependent variables, Y s will be shaped as $[\psi(N_{input}, N_{output}), 7, 10]$

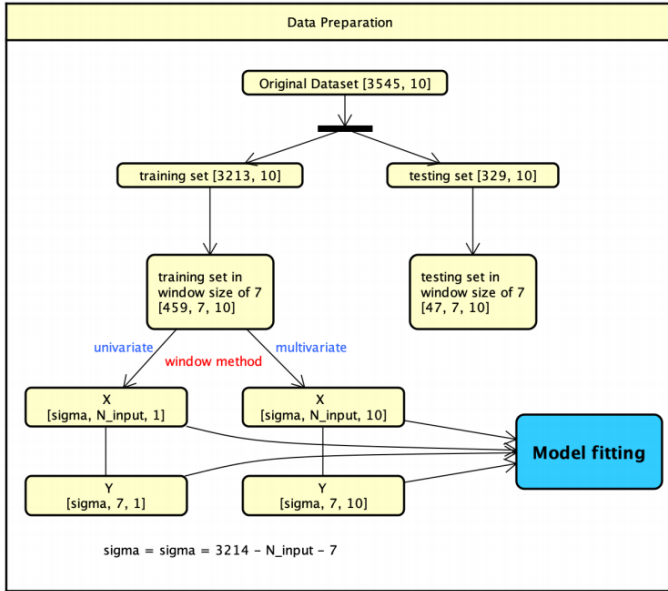


Fig. 8 Data Preparation Procedure with Window Method For a 7-day Forecast

Test Harness: a baseline persistence forecast

Now the data is prepared and the following procedure is to build a baseline model to measure the validity, performance and improvements of our LSTM models. If our models achieve performance below the baseline, the model needs to be fixed or simply abandoned. A persistence

forecast, or naive forecast, is a simple, fast and repeatable method which assumes little about the nature of the forecast (Hyndman and Athanasopoulos 2018). It can be generalized to testing possible offsets of the historical data. For instance, persisting last week's measurements, persisting last year's measurements and etc. Three models were constructed using different offsets. Model one, the daily persistence, uses exact measurement from one day before our forecasting period and applies this value to all forecasting period as predictions. Model two, the weekly persistence, uses the measurements one week prior to our forecasting period as predictions to the forecasting period. Model three, the weekly one-year ago persistence, uses a year-ago's measurements and taking one week's measurements prior to the identical week as our forecasting period. An example of 7-day forecast persistence models are demonstrated in Fig. 9.

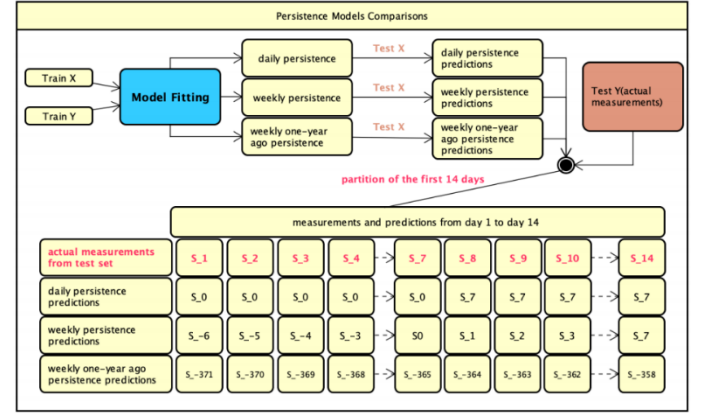
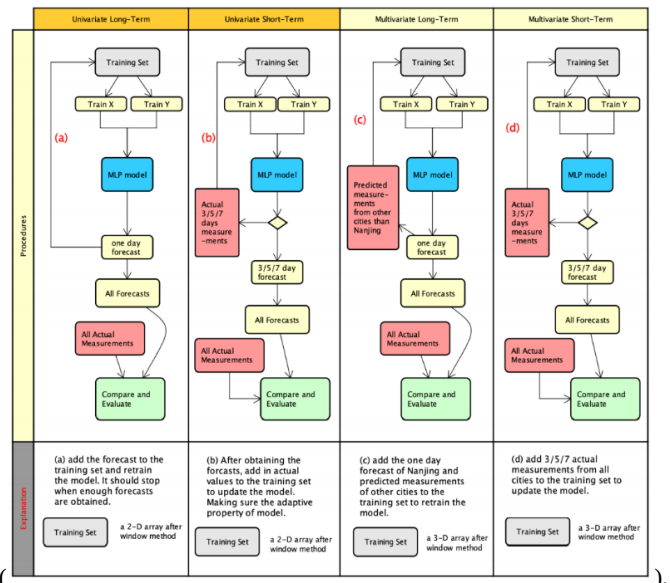


Fig. 9 Persistence Models Comparisons For a 7-day Forecast

Model Construction

After constructing the baseline persistence models, we start building skillful LSTM models. A skillful model is a model that performs better than the baseline model (Brownlee 2018). The reason to build a skillful model first, instead of blatantly tuning parameters, is to confirm the validation of the model. We don't expect the models to be the best possible but we are expecting to deliver a model that's around 80% of what can be possibly achieved. As stated in the problem definition, we will build two main kinds of models, namely univariate and multivariate, for both long-term and short-term forecast



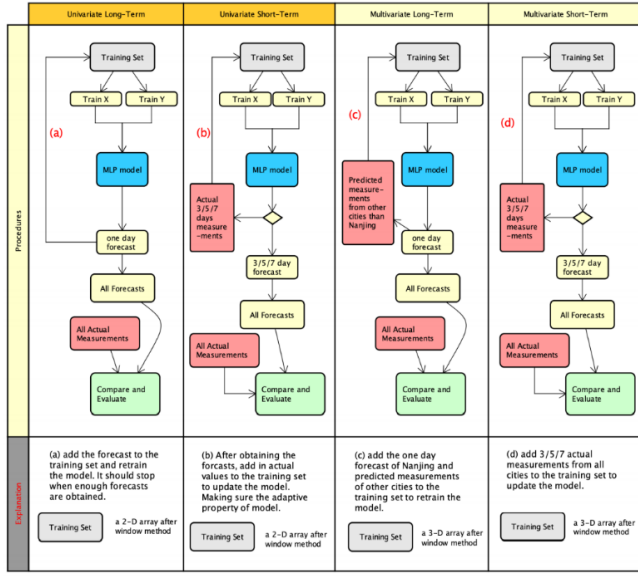


Fig. 10 Four Different LSTM models for Four Scenarios

For the univariate case, only Changjiang water's water level data will be used, i.e., we will discard all other cities' measurements. The mechanics used for long-term forecast and short-term forecast are quite disparate. In terms of long-term forecast, we will start by applying window method on our training set and obtain the reconstructed data frames, namely Train X and Train Y. Then utilizes these two data frames to fit our LSTM univariate model and expect a one-day forecast when feeding the neural network with historical values in the Train X as inputs. Additionally, adding this one-day forecast to the original dataset and retrain our model, then feeding it with historical values to make another one-day forecast. Repeating this process until sufficient forecasts have been obtained, i.e., same number of observations as the testing set. Lastly, we can compare our forecasts with the actual measures point by point and calculate the error in metrics of RMSE. On the other hand, for short-term forecast, the main difference is that forecasts are not added to the training set but actual measurements are added in order to update the model, resulting in an adaptive model that changes dynamically based on the actual inputs.

For multivariate case, all cities' water level will be used. These models are expected to outperform the univariate models as neural networks have the ability to distinguish complex mappings and the relationships (upstream and downstream relations) between other cities. The procedure is slightly adjusted from the univariate case but still adapting the idea. For long-term forecast, repeating one-step forecast and update the model using Changjiang's one-day forecast along with actual measurement from other cities. For short-term forecast, we are now updating the model with actual measurements from all the cities, also resulting in an adaptive model that changes dynamically based on the actual inputs. One other thing to notice is that after applying the rolling window method to the training set with multiple variables, the resulting data frame will be 3-D instead of 2-D as these variables occupy a dimension. This adaptive nature of model is valuable in real life. For instance, if there are unpredictable factors heavily influencing today's water-level, an adaptive model can quickly adapt this change and update the model for future accurate predictions.

Parameter Tuning

After determining the skillfulness of the models constructed earlier, we may think about ways to improve the performance or achieving the best performance of the models by tuning parameters. Parameters are

important components of neural networks. In this study, the parameters selections are: N_{input} , N_{nodes} , N_{epochs} , N_{batch} , N_{hidden} .

For example, if we would like to create a univariate LSTM model for a 7-day forecast (Fig. 11). We start by setting N_{input} in order to prepare the data using window method. After that, the inputs and outputs of the neural network are now defined. We then decide how many hidden layers we are using, i.e., N_{hidden} and for each hidden layer, how many nodes(neurons) to use, i.e., N_{nodes} . Now the structure of LSTM is established, we can start training the model. Picking the number of times, we want to repeat this process back and forth, i.e., N_{epochs} , and decide how many samples are thrown 214 into one mini batch, i.e., N_{batch} .

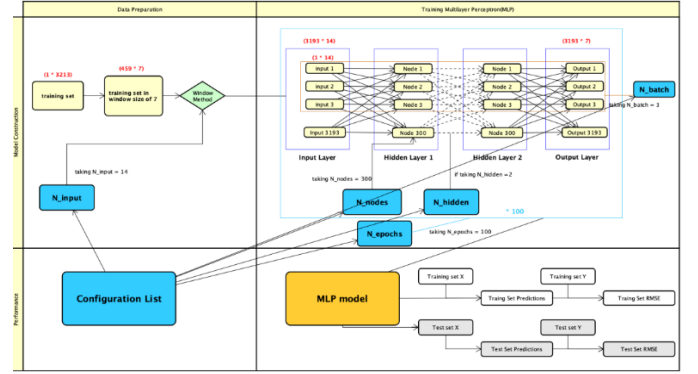


Fig. 11 An example of how parameters are used in the process of creating a Univariate LSTM model for a 7-day Forecast. Repeat until all combinations of parameters are tested for performance.

Parameter tuning is lengthy and computationally costly. Luckily, with the assistance of high-performance computing clusters (HPCCs), the issue is resolved. Hence, we built a grid search network that takes into all combinations of the parameters into account. After each run of the configurations, we will re-run with the same set of configurations to ensure it's not just luck due to the stochastic nature of neural networks. RMSEs of the testing set and training set from all runs were measured and were divided by the number of runs as the metrics of the model performance.

RESULTS

Short-term Forecasts

The short-term forecasts consist of 3-, 5- and 7-days forecasting. Each scenario consists of the baseline(persistence) model, univariate LSTM and multivariate LSTM. For univariate LSTM and multivariate LSTM, their model configurations are selected by the grid search network constructed earlier using high performance computing clusters. To simplify the presentation, only the top three configurations of each model will be reported and the best configuration's actual v.s. predicted will be plotted. We will use 3213 days as training set and make a prediction with a model created with the training set for the next 3/5/7 days, then add the actual 3/5/7 days' measurements to the training set and update the model. Repeat this process and record each prediction cumulatively until the predictions reach the length of the testing set. Each combination of configurations was repeatedly evaluated ten times and each time a measurement of error, in specific, RMSE, was collected and taken a mean as the final RMSE of the model.

In all cases (3,5- and 7-days forecasts), multivariate LSTM outperformed

other models by a lot as we expected (Tab. 1 ~ Tab. 3), given the non-negligible relationships between different cities. For 3-day forecast, univariate LSTM and multivariate LSTM produced predictions 35.0% and 45.8% lower in terms of RMSE. For 5-day forecast, univariate LSTM and multivariate LSTM produced predictions 30.1% and 43.9% lower in terms of RMSE. For 7-day forecast, univariate LSTM and multivariate LSTM produced predictions 27.6% and 48.2% lower in terms of RMSE. Multilayer perceptron or LSTM have shown great performance in short-term time series forecasts for waterways' water-level fluctuations, as shown in Fig. 12 ~ Fig. 14. The statistical performances are shown in Tab. 4.

	Model	Top 3 Configurations	RMSE	Best Model Prediction vs actual plot
3-day forecast	Persistence (baseline)	Daily Persisted	0.297	
		Weekly Persisted	0.903	
		Weekly one-year-ago persisted	0.934	
	Univariate MLP	200, 1000, 200, 6, 1	0.193	
		200, 1000, 200, 10, 1	0.201	
		200, 3000, 200, 6, 1	0.207	
	Multivariate MLP	150, 700, 150, 6, 1	0.161	
		100, 700, 150, 10, 1	0.176	
		100, 700, 100, 6, 1	0.177	

Tab. 1 3-day forecast results with top 3 configurations in each case

	Model	Top 3 Configurations	RMSE	Best Model Prediction vs actual plot
5-day forecast	Persistence (baseline)	Daily Persisted	0.362	
		Weekly Persisted	1.148	
		Weekly one-year-ago persisted	1.138	
	Univariate MLP	200, 1700, 200, 6, 1	0.253	
		200, 2000, 200, 6, 1	0.263	
		150, 500, 150, 10, 1	0.275	
	Multivariate MLP	150, 1000, 200, 6, 1	0.203	
		150, 700, 130, 6, 1	0.218	
		200,1700, 200, 5, 1	0.220	

Tab. 2 5-day forecast results with top 3 configurations in each case

	Model	Top 3 Configurations	RMSE	Best Model Prediction vs actual plot
7-day forecast	Persistence (baseline)	Daily Persisted	0.442	
		Weekly Persisted	1.307	
		Weekly one-year-ago persisted	0.803	
	Univariate MLP	150, 500, 200, 7, 1	0.320	
		150, 1000, 200, 10, 1	0.322	
		150, 700, 200, 7, 1	0.324	
	Multivariate MLP	150, 500, 200, 7, 1	0.229	
		200, 700, 150, 7, 1	0.245	
		100, 100, 100, 7, 1	0.250	

Tab. 3 7-day forecast results with top 3 configurations in each case

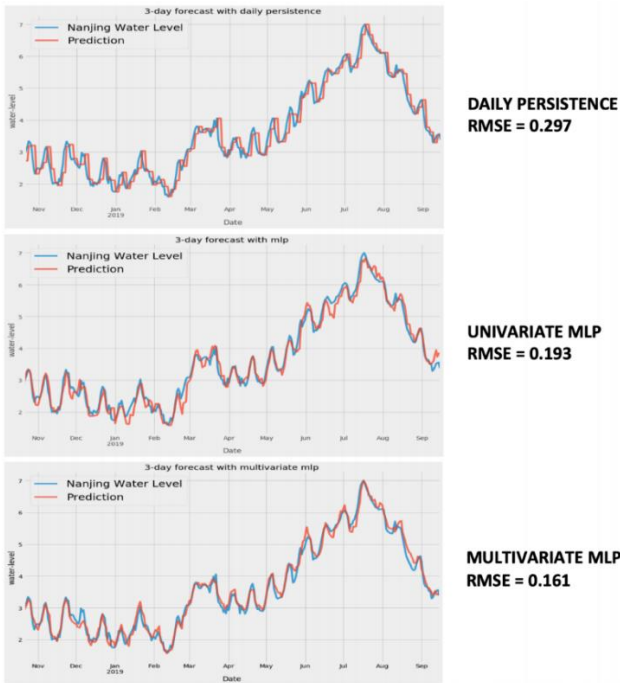


Fig. 12 A zoomed-in comparisons of three best models for 3-day forecast

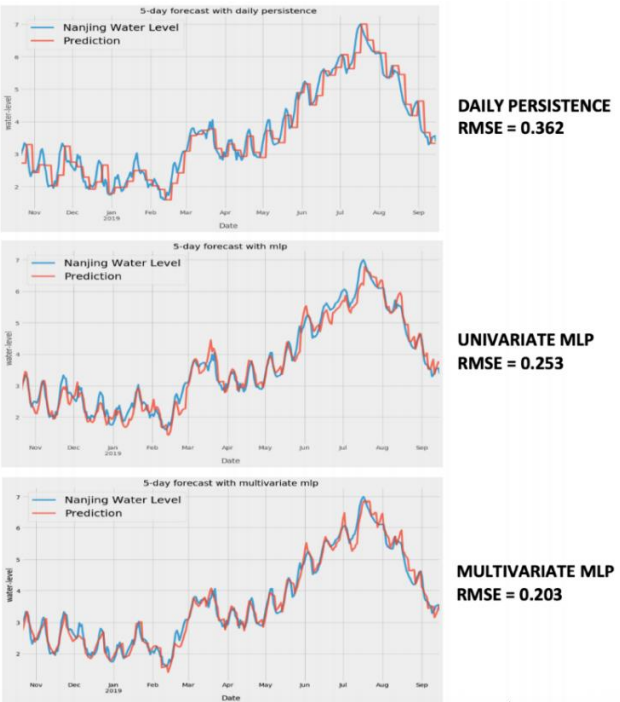
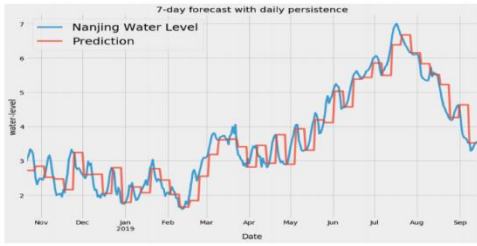
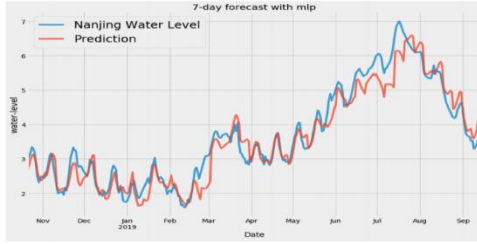


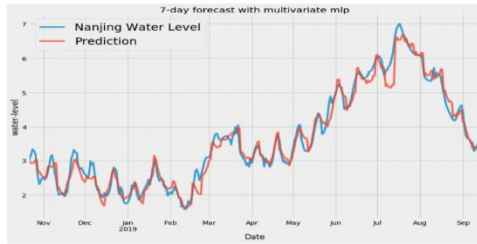
Fig. 13 A zoomed-in comparisons of three best models for 5-day forecast



DAILY PERSISTENCE
RMSE = 0.442



UNIVARIATE MLP
RMSE = 0.320



MULTIVARIATE MLP
RMSE = 0.229

Fig. 14 A zoomed-in comparisons of three best models for 7-day forecast

	Model	RMSE	Relative RMSE to the baseline	Relative Lower RMSE to the baseline
3 day	Baseline	0.297	100%	0%
	Univariate MLP	0.193	65.0%	35%
	Multivariate MLP	0.161	54.2%	45.8%
5 day	Baseline	0.362	100%	0%
	Univariate MLP	0.253	69.9%	30.1%
	Multivariate MLP	0.203	56.1%	43.9%
7 day	Baseline	0.442	100%	0%
	Univariate MLP	0.320	72.4%	27.6%
	Multivariate MLP	0.229	51.8%	48.2%

Tab. 4 Performance Measurements of models in short-term forecast

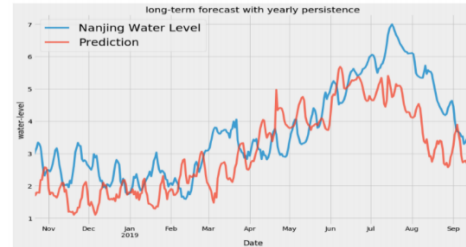
Long-term Forecast

The long-term forecast is a forecast with the length of the testing set. In other words, we are predicting the full sequence of time series whose length is identical to the testing set's. In this study we are predicting a full sequence of time series of length 330. Long-term forecast is less likely to be accurate but it offers a trend which be valuable in real life setting. The yearly persistence model, i.e., using last year's same 330 days' measurements, is constructed as the baseline model to evaluate the performance of the LSTM models. The univariate LSTM model and multivariate LSTM model are constructed using the training sets. The mechanics used in univariate LSTM and multivariate LSTM are analogous. In univariate LSTM, first we obtain one forecast from the constructed model and add this one forecast to the training set to construct a new model, then obtaining another forecast. Repeat this process until we obtain enough forecasts whose length matches the test set's. In multivariate LSTM, first we obtain 10 forecasts for 10 cities from the constructed model and add these 10 forecasts to the training set

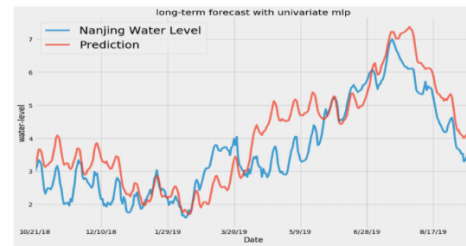
to construct a new model, then obtaining another 10 forecasts. Repeat this process until we obtain enough forecasts whose length matches the test set's. Upon gaining all the forecasts, we measure the performance of the model by calculate RMSE and repeat the whole process for 10 times due to the stochastic nature of neural networks. The final RMSE is calculated by the mean of cumulative RMSE by each trial. The results are shown in Tab. 6 and the zoom-in comparisons are shown in Fig. 15 For brevity, we will only include the top three configurations. Consequently, Multivariate LSTM again outperformed all the other models. Univariate LSTM showed a 25.0% decrease in RMSE and multivariate LSTM showed a 66.2% decrease in RMSE compared to the baseline model, as shown in Tab. 5.

Model	Top 3 Configurations	RMSE	Best Model Prediction vs actual plot
Persistence (baseline)	Yearly Persisted	1.063	
Univariate MLP	800, 600, 200, 100, 1	0.797	
	800, 1000, 300, 20, 1	0.851	
	700, 2000, 200, 50, 1	0.852	
Multivariate MLP	750, 2000, 200, 230, 1	0.359	
	700, 1700, 150, 300, 1	0.380	
	1000, 800, 150, 300, 1	0.399	

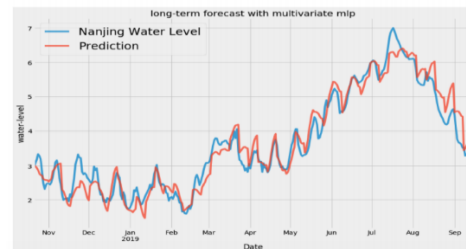
Tab. 6 Long-term forecast results



DAILY PERSISTENCE
RMSE = 1.063



UNIVARIATE MLP
RMSE = 0.797



MULTIVARIATE MLP
RMSE = 0.359

Fig. 15 A zoomed-in comparisons of three best models for long-term forecast

Model	RMSE	Relative RMSE to the baseline	Relative Lower RMSE to the baseline
Baseline	1.063	100%	0%
Univariate MLP	0.797	75.0%	25.0%
Multivariate MLP	0.359	33.8%	66.2%

Tab. 5 Performance Measurements of models in long-term forecast

DISCUSSIONS AND CONCLUSIONS

From the results, the power of multilayer perceptron for time series forecasting has been perfectly demonstrated. For short-term forecast, univariate LSTM models have shown a significant improvement when compared to the baseline model, giving a fairly accurate forecast for 3/5/7 days forecasting. Additionally, multivariate LSTM models provided even more improvements than univariate LSTM models. For long-term forecast, the performance of univariate LSTM models is poor, only being a little better than the baseline model. However, the trend and seasonality can be somehow spotted. On the other hand, multivariate LSTM models performed outstandingly as the effect between each waterway is non-negligible. Multivariate LSTM provided a valuable full sequence forecast of 330 days only with a RMSE of 0.359 resulting in a final deliverable model that can be utilized in governmental waterway regulations. Overall, it's a success of experimenting ANNs on waterways' WLFs for the first time and the models are valuable in real-life settings.

However, further improvements can be made using convolutional neural networks (CNNs), recurrent neural networks (RNNs) and their mixtures due to the limitations of LSTM (Brownlee 2018). For water-level fluctuations, the data of which contains strong factors like trend and seasonality, providing a meaningful mapping for inputs and outputs of LSTMs and making it valid for water-level forecasts. But LSTM lacks the ability to perform feature learning which is a key attribute of CNNs and RNNs and the ability to learn temporal dependence as RNNs (Brownlee 2018). Additionally, due to the fully connected nature of LSTM, it's quite computationally costly. The whole process of grid-searching using high performance computing clusters took a total of 170 hours and still could not cover every combination of configurations. To resolve these issues, we can construct univariate and multivariate CNNs, RNNs and their mixtures, which are more computationally economic and have better ability to perform well for time series problems.

ACKNOWLEDGEMENT

This project is sponsored by the National Natural Science Foundation of China (Grant No.51879058) and Qing Lan Project.

REFERENCES

- Altunkaynak, A., Özger, M., and Sen, Z. (2003). "Triple diagram model of level fluctuations in lake van, turkey
- Bontempi, G., Taieb, S. B., and Le Borgne, Y.-A. (2012). "Machine learning strategies for time series forecasting." *European business intelligence summer school*, Springer, 62–77.
- Brownlee, J. (2018). *Deep Learning for Time Series Forecasting: Predict the Future with LSTMs, CNNs and LSTMs in Python*. Machine Learning Mastery.
- Driss, S. B., Soua, M., Kachouri, R., and Akil, M. (2017). "A comparison study between LSTM and convolutional neural network models for character recognition." *Real-Time Image and Video Processing 2017*, Vol. 10223, International Society for Optics and Photonics, 1022306.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep learning*. MIT press.
- Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The elements of statistical learning: data mining, inference, and prediction*. Springer Science & Business Media.
- Haykin, S. (1994). *Neural networks: a comprehensive foundation*. Prentice Hall PTR.
- Heaton, J. (2008). *Introduction to neural networks with Java*. Heaton Research, Inc.
- Hecht-Nielsen, R. (1992). "Theory of the backpropagation neural network." *Neural networks for perception*, Elsevier, 65–93.
- Hyndman, R. J. and Athanasopoulos, G. (2018). *Forecasting: principles and practice*. OTexts.
- Jonkeren, O., Jourquin, B., and Rietveld, P. (2011). "Modal-split effects of climate change: The effect of low water levels on the competitive position of inland waterway transport in the river rhine area." *Transportation Research Part A: Policy and Practice*, 45(10), 1007–1019.
- Li, L., Noorian, F., Moss, D. J., and Leong, P. H. (2014). "Rolling window time series prediction using map reduce." *Proceedings of the 2014 IEEE 15th International Conference on Information Reuse and Integration (IEEE IRI 2014)*, IEEE, 757–764.
- Piasecki, A., Jurasz, J., and Skowron, R. (2017). "Forecasting surface water level fluctuations of lake serwy (northeastern poland) by artificial neural networks and multiple linear regression." *Journal of Environmental Engineering and Landscape Management*, 25(4), 379–388.
- Ramachandran, P., Zoph, B., and Le, Q. V. (2017). "Searching for activation functions." *arXiv preprint arXiv:1710.05941*.
- Rosenblatt, F. (1961). "Principles of neurodynamics. perceptrons and the theory of brain mechanisms." *Report no.*, Cornell Aeronautical Lab Inc Buffalo NY.
- Svozil, D., Kvasnicka, V., and Pospichal, J. (1997). "Introduction to multi-layer feed-forward neural networks." *Chemometrics and intelligent laboratory systems*, 39(1), 43–62.
- Tsay, R. S. (2013). *Multivariate time series analysis: with R and financial applications*. John Wiley & Sons.